

Atelier 3

Docker & La livraison Continue

Conteneurisation • Dockerfile • Docker Compose

DevOps - UP ASI

Bureaux E204 | E304



À l'issue de cet atelier, l'étudiant sera capable de :

- À l'issue de cet atelier, l'étudiant sera capable de :
- Vérifier et utiliser Docker sur une VM Ubuntu
- Maîtriser les commandes Docker essentielles (*run, pull, ps, stop, rm...*)
- Créer une image Docker personnalisée via un Dockerfile
- Lancer et tester un conteneur depuis une image locale
- Orchestrer plusieurs conteneurs avec Docker Compose
- Déployer une stack Spring Boot + MySQL via Docker Compose
- Travail à faire : Intégrer Docker dans un pipeline Jenkins (*build + push image*)

Pour travailler cet atelier il faut que :

- Atelier 2 soit complété (*Jenkins installé et opérationnel*).
- VM Ubuntu soit démarrée : *vagrant up*
- Soit connecté à la VM : *vagrant ssh*
- Script `install_devops_tools.sh` exécuté (*Docker installé*)
- Jenkins accessible : `http://192.168.33.10:8080`

► Vérification de l'environnement Docker

```
vagrant@ubuntu - Terminal

1 — Vérifier la version Docker

1 vagrant@ubuntu:~$ docker --version

Résultat attendu :
Docker version 24.x.x, build xxxxxxx
```

Vérifie que Docker est bien **installé** sur la machine. Si la commande n'est pas reconnue, Docker n'est pas installé ou n'est pas dans le PATH. C'est toujours la première commande à lancer avant de travailler avec Docker.

```
vagrant@ubuntu - Terminal

2 — Vérifier que Docker tourne

1 vagrant@ubuntu:~$ sudo systemctl status docker

Résultat attendu :
● docker.service - Docker Application Container Engine
Active: active (running)
```

Vérifie que le service Docker tourne en arrière-plan. Docker peut être installé mais pas démarré. Si le statut est inactive ou failed, les commandes Docker ne fonctionneront pas. Cette commande permet de diagnostiquer rapidement si Docker est opérationnel.

► Vérification de l'environnement Docker



vagrant@ubuntu - Terminal

3 — Vérifier les images disponibles

```
1 vagrant@ubuntu:~$ docker images
```

REPOSITORY TAG SIZE

sonarqube	latest	640MB
sonatype/nexus3	latest	720MB
prom/prometheus	latest	245MB
grafana/grafana	latest	380MB

Liste toutes les images Docker disponibles localement sur la machine. Une image est le modèle à partir duquel on crée des conteneurs. Grâce au script `install_devops_tools.sh`, les images SonarQube, Nexus, Prometheus et Grafana sont déjà téléchargées, pas besoin de les re-télécharger à chaque utilisation.

vagrant@ubuntu - Terminal

4 — Éviter sudo devant chaque commande Docker

```
1 vagrant@ubuntu:~$ sudo usermod -aG docker vagrant
2 vagrant@ubuntu:~$ newgrp docker
```

⚠ Important :

Après cette commande, plus besoin de sudo devant chaque commande Docker.

Par défaut, seul root peut exécuter les commandes Docker. Ces deux commandes ajoutent l'utilisateur vagrant au groupe Docker pour pouvoir utiliser Docker sans sudo. Sans ça, chaque commande Docker nécessiterait `sudo docker ...` ce qui est fastidieux et risqué en production.

▶ Commandes Docker essentielles

```
vagrant@vagrant: ~  
vagrant@vagrant:~$ docker run hello-world  
Unable to find image 'hello-world:latest' locally  
latest: Pulling from library/hello-world  
4f55086f7dd0: Pull complete  
d5e71e642bf5: Download complete  
Digest: sha256:96498ffd522e70807ab6384a5c0485a79b9c7c08ca79ba08623edcad1054e62d  
Status: Downloaded newer image for hello-world:latest  
  
Hello from Docker!  
This message shows that your installation appears to be working correctly.
```

Lancer un premier conteneur

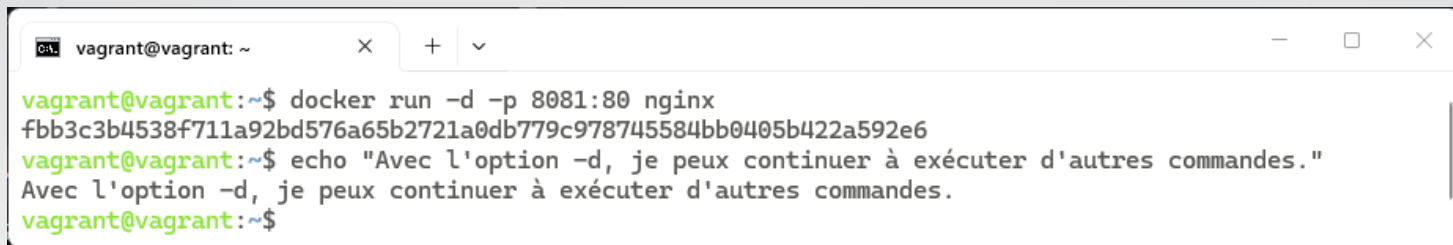
C'est le "**Hello World**" de Docker. Docker télécharge une toute petite image depuis Docker Hub, crée un conteneur, affiche un message de confirmation et s'arrête. Sert uniquement à vérifier que Docker fonctionne correctement de bout en bout.

Commandes Docker essentielles

```
vagrant@vagrant: ~  
vagrant@vagrant:~$ docker run -it ubuntu bash  
Unable to find image 'ubuntu:latest' locally  
latest: Pulling from library/ubuntu  
d1f56e4c7f2f: Pull complete  
81e2f2053c8f: Pull complete  
107e4f1717f2: Download complete  
Digest: sha256:53958ec7b67c2c9355df922dd08dbf0360611f8c3cdb656875e81873db9ffdba  
Status: Downloaded newer image for ubuntu:latest  
root@d98ad386a79b:/# ls  
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var  
root@d98ad386a79b:/# exit  
exit  
vagrant@vagrant:~$ |
```

Lancer un conteneur Ubuntu interactif

Normalement un conteneur lance une commande et s'arrête. Avec **-it** tu ouvres un terminal à l'intérieur du conteneur, exactement comme si tu te connectais à une machine distante via SSH. Tu peux naviguer dans les fichiers, installer des logiciels, tester des commandes... tout ça dans un environnement isolé de ta VM.



```
vagrant@vagrant: ~  
vagrant@vagrant:~$ docker run -d -p 8081:80 nginx  
fbb3c3b4538f711a92bd576a65b2721a0db779c978745584bb0405b422a592e6  
vagrant@vagrant:~$ echo "Avec l'option -d, je peux continuer à exécuter d'autres commandes."  
Avec l'option -d, je peux continuer à exécuter d'autres commandes.  
vagrant@vagrant:~$
```

Lancer un conteneur en arrière-plan

-d = detached : le conteneur tourne en arrière-plan, tu récupères la main dans ton terminal

-p 8081:80 = port mapping : relie le port 8081 de ta VM au port 80 du conteneur

Sans -d le conteneur bloquerait ton terminal jusqu'à ce que tu l'arrêtes

```
vagrant@vagrant: ~  
vagrant@vagrant:~$ docker ps  
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES  
fbb3c3b4538f   nginx    "/docker-entrypoint...."  11 minutes ago Up 11 minutes  0.0.0.0:8081->80/tcp, [::]:8081->80/tcp  jovial_lamport
```

Affiche les conteneurs actuellement en cours d'exécution.

Montre l'ID, le nom de l'image, le port, le statut et la durée.

```
vagrant@vagrant: ~$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
e85fd31a47a3	ubuntu	"bash"	4 minutes ago	Exited (0) 4 minutes ago		sad_lederberg
7e2367f220db	hello-world	"/hello"	8 minutes ago	Exited (0) 8 minutes ago		nice_nash
5d9d6039591f	ubuntu	"bash"	8 minutes ago	Exited (0) 8 minutes ago		sleepy_perlman
fbb3c3b4538f	nginx	"/docker-entrypoint...."	15 minutes ago	Up 15 minutes	0.0.0.0:8081->80/tcp, [::]:8081->80/tcp	jovial_lamport

Affiche tous les conteneurs, y compris ceux qui sont arrêtés.

Utile pour voir l'historique des conteneurs et récupérer leur ID pour les supprimer.

```

vagrant@vagrant: ~
vagrant@vagrant:~$ docker stop fbb3c3
fbb3c3
vagrant@vagrant:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS              PORTS          NAMES
e85fd31a47a3   ubuntu        "bash"                  7 minutes ago  Exited (0) 7 minutes ago           sad_lederberg
7e2367f220db   hello-world   "/hello"                11 minutes ago  Exited (0) 11 minutes ago           nice_nash
5d9d6039591f   ubuntu        "bash"                  11 minutes ago  Exited (0) 11 minutes ago           sleepy_perlman
fbb3c3b4538f   nginx         "/docker-entrypoint...." 18 minutes ago  Exited (0) 8 seconds ago           jovial_lamport

```

Affiche Arrête proprement un conteneur en cours d'exécution.

Le conteneur existe toujours sur le disque mais ne tourne plus. Comme éteindre une machine sans la supprimer.

```
vagrant@vagrant: ~  
vagrant@vagrant:~$ docker rm fbb3c3  
fbb3c3  
vagrant@vagrant:~$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
e85fd31a47a3	ubuntu	"bash"	9 minutes ago	Exited (0) 9 minutes ago		sad_lederberg
7e2367f220db	hello-world	"/hello"	14 minutes ago	Exited (0) 14 minutes ago		nice_nash
5d9d6039591f	ubuntu	"bash"	14 minutes ago	Exited (0) 14 minutes ago		sleepy_perlman

Supprimer définitivement un conteneur arrêté.

Libère l'espace disque. Le conteneur doit être arrêté avant d'être supprimé (*sinon utiliser docker rm -f*).

```
vagrant@vagrant: ~  
vagrant@vagrant:~$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
grafana/grafana:latest	5dad0df181cb	1.47GB	352MB	
hello-world:latest	96498ffd522e	25.9kB	9.49kB	U
nginx:latest	ec4ed8b5299e	241MB	66MB	
prom/prometheus:latest	a75c5a35bc21	427MB	118MB	
sonarqube:latest	4671314254b0	2.53GB	1.08GB	
sonatype/nexus3:latest	20eff0386283	1.13GB	469MB	
ubuntu:latest	53958ec7b67c	160MB	45.3MB	U

Info → U In Use

Lister toutes les images Docker disponibles localement sur ta machine.

Petit Rappel : L'image Docker est la classe en POO et le conteneur Docker est l'objet instanciée depuis cette classe.

```
vagrant@vagrant: ~$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
grafana/grafana:latest	5dad0df181cb	1.47GB	352MB	
hello-world:latest	96498ffd522e	25.9kB	9.49kB	
nginx:latest	ec4ed8b5299e	241MB	66MB	
prom/prometheus:latest	a75c5a35bc21	427MB	118MB	
sonarqube:latest	4671314254b0	2.53GB	1.08GB	
sonatype/nexus3:latest	20eff0386283	1.13GB	469MB	
ubuntu:latest	53958ec7b67c	160MB	45.3MB	U

```
vagrant@vagrant:~$ docker rmi hello-world
Untagged: hello-world:latest
Deleted: sha256:96498ffd522e70807ab6384a5c0485a79b9c7c08ca79ba08623edcad1054e62d
vagrant@vagrant:~$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
grafana/grafana:latest	5dad0df181cb	1.47GB	352MB	
nginx:latest	ec4ed8b5299e	241MB	66MB	
prom/prometheus:latest	a75c5a35bc21	427MB	118MB	
sonarqube:latest	4671314254b0	2.53GB	1.08GB	
sonatype/nexus3:latest	20eff0386283	1.13GB	469MB	
ubuntu:latest	53958ec7b67c	160MB	45.3MB	U

Supprime une image Docker localement.

Libère de l'espace disque. Impossible de supprimer une image si un conteneur l'utilise encore.

Le Dockerfile - Créer une image avec un Dockerfile

```
vagrant@vagrant: ~/mc X + - □ X
vagrant@vagrant:~$ mkdir ~/mon-image
vagrant@vagrant:~$ cd ~/mon-image
vagrant@vagrant:~/mon-image$
```

Créer un dossier de travail

```
vagrant@vagrant: ~/mon-im X + - □ X
GNU nano 6.2 script.sh
#!/bin/sh
echo "=====
echo "Bonjour depuis mon conteneur Docker !"
echo "Date : $(date)"
echo "Hostname : $(hostname)"
echo "=====
[ Wrote 6 lines ]
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify
```

Créer un script shell

```
vagrant@vagrant: ~/mon-im X + - □ X
GNU nano 6.2 Dockerfile *
FROM alpine:3.18
WORKDIR /app
COPY script.sh /app/script.sh
RUN chmod +x /app/script.sh
CMD ["/bin/sh", "/app/script.sh"]
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify
```

Créer le Dockerfile

Le Dockerfile - Créer une image avec un Dockerfile

```
vagrant@vagrant: ~/mon-image$ docker build -t mon-image:1.0 .
[+] Building 4.9s (9/9) FINISHED    docker:default
=> [internal] load build definition from Dockerfile    0.0s
=> => transferring dockerfile: 159B    0.0s
=> [internal] load metadata for docker.io/library/alpi  2.4s
=> [internal] load .dockerignore    0.0s
=> => transferring context: 2B    0.0s
=> [1/4] FROM docker.io/library/alpine:3.18@sha256:de0 1.8s
=> => resolve docker.io/library/alpine:3.18@sha256:de0 0.0s
=> => sha256:44cf07d57ee4424189f012074 3.42MB / 3.42MB 1.6s
=> => extracting sha256:44cf07d57ee4424189f012074a5911 0.1s
=> [internal] load build context    0.0s
=> => transferring context: 249B    0.0s
=> [2/4] WORKDIR /app    0.1s
=> [3/4] COPY script.sh /app/script.sh    0.0s
=> [4/4] RUN chmod +x /app/script.sh    0.2s
=> => exporting to image    0.2s
=> => exporting layers    0.1s
=> => exporting manifest sha256:babfd64dab0943b40e9985 0.0s
=> => exporting config sha256:ebb9bb92a01bef2efa560617 0.0s
=> => exporting attestation manifest sha256:8548663db0 0.0s
=> => exporting manifest list sha256:cc7a11fff72760110 0.0s
=> => naming to docker.io/library/mon-image:1.0    0.0s
=> => unpacking to docker.io/library/mon-image:1.0    0.0s
```

Construire l'image

```
vagrant@vagrant: ~/mon-image$ docker run mon-image:1.0
=====
Bonjour depuis mon conteneur Docker !
Date : Fri Jun 26 17:18:02 UTC 2026
Hostname : af7dc5b3fc30
=====
```

Lancer le conteneur

En Docker, le choix de l'image de base est crucial. Une image trop lourde impacte :

- Le temps de build (*plus long*)
- Le temps de déploiement (*plus lent à télécharger*)
- La sécurité (*plus de packages = plus de vulnérabilités potentielles*)

La règle d'or en Docker : Utiliser l'image la plus petite possible qui répond à ton besoin.

Pourquoi Alpine spécifiquement ?

Alpine Linux est une distribution Linux minimaliste basée sur musl libc et BusyBox. Elle a été conçue dès le départ pour être légère et sécurisée.

- Elle ne contient que le strict minimum pour faire tourner une application
- Pas de services inutiles, pas de packages superflus
- Chaque package supplémentaire s'installe via apk add
- C'est la base de la majorité des images officielles Docker (Node, Python, Go...)

► Bonne pratique : Choisir la bonne image de base

Images Docker — Choisir la bonne base

Image	Taille	Cas d'usage
ubuntu:22.04	~77 MB	Complète, facile mais lourde
debian:slim	~80 MB	Légère, compatible apt
alpine:3.18	~5 MB ✓	Ultra légère, populaire — recommandée
busybox	~1.5 MB	Minimale, juste un shell

Pourquoi Alpine est la plus utilisée en production ?

Ultra légère 5 MB vs 77 MB pour Ubuntu	Sécurisée Surface d'attaque minimale	Gestionnaire apk apk add à la place de apt-get
---	---	---

Si tu déploies 100 conteneurs sur un cluster avec Ubuntu (77MB) ça sera **7.7 GB** à télécharger
 et avec Alpine (5MB) → **500 MB** à télécharger.

La différence est énorme en production !

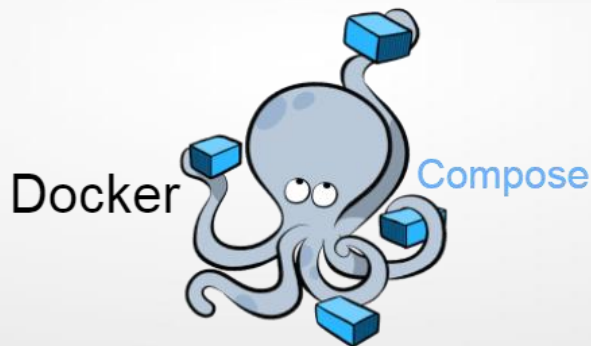
C'est quoi Docker Compose ?

Docker Compose permet de définir et lancer plusieurs conteneurs en même temps via un fichier docker-compose.yml.

Au lieu de lancer manuellement chaque conteneur, une seule commande suffit.

Cas d'usage

Application Spring Boot qui a besoin de MySQL, 2 conteneurs à lancer ensemble et à faire communiquer.



▶ Le Dockerfile - Créer une image avec un Dockerfile

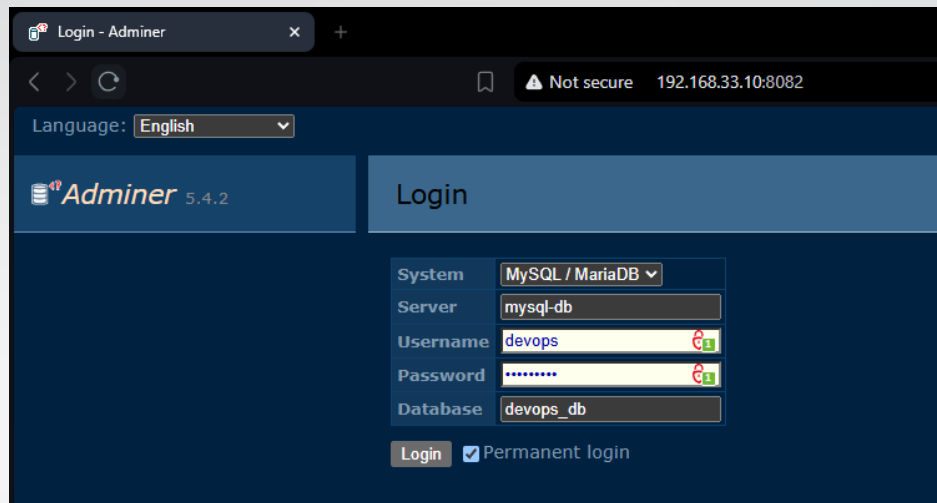
```
vagrant@vagrant: ~/ma × + - □ ×  
vagrant@vagrant:~$ mkdir ~/ma-stack  
vagrant@vagrant:~$ cd ma-stack  
vagrant@vagrant:~/ma-stack$
```

Créer un dossier de travail

```
vagrant@vagrant: ~/ma × + - □ ×  
GNU nano 6.2 docker-compose.yml *  
version: '3.8'  
  
services:  
  
  mysql:  
    image: mysql:8.0  
    container_name: mysql-db  
    environment:  
      MYSQL_ROOT_PASSWORD: root  
      MYSQL_DATABASE: devops_db  
      MYSQL_USER: devops  
      MYSQL_PASSWORD: devops123  
    ports:  
      - "3306:3306"  
    volumes:  
      - mysql-data:/var/lib/mysql  
  
  adminer:  
    image: adminer  
    container_name: adminer-ui  
    ports:  
      - "8082:8080"  
    depends_on:  
      - mysql  
  
volumes:  
  mysql-data:|  
  
^G Help ^O Write Out ^W Where Is ^K Cut  
^X Exit ^R Read File ^L Replace ^U Paste
```

Créer le docker-compose.yml et faire *docker compose up -d*

▶ Le Dockerfile - Créer une image avec un Dockerfile



Accéder à Adminer (interface web MySQL) sur <http://192.168.33.10:8082>

Serveur : mysql-db

Utilisateur : devops

Mot de passe : devops123

Base : devops_db

On peut arrêter le stack avec *docker compose down*

Objectif

Conteneuriser et orchestrer une application complète (*Frontend + Backend + Base de données*) avec Docker Compose, puis intégrer le tout dans un pipeline Jenkins.

Vous pouvez retrouver le lien du projet pour le recloner sous :

<https://github.com/Alaa-Rami/DevOps-AppGestionDesProjets.git>

Contraintes :

- Le Backend doit démarrer après la base de données (*depends_on*)
- Le Frontend doit démarrer après le Backend (*depends_on*)
- La base de données doit avoir un volume persistant
- Les services doivent communiquer via un réseau Docker dédié



Si vous avez des questions
n'hésitez pas à nous contacter



Département Informatique
UP Architectures des Systèmes d'Information
Bureaux E204 | E304

Module DevOps – 2026 - 2027